

Тема 1. Семинар: повторение на практике — компоненты, сервер, фильтр

Организационное примечание. Семинар и лекция по этой теме проходят в один день, и порядок может отличаться. Задания составлены так, что их можно выполнять и до лекции: всё необходимое было пройдено в прошлом семестре, а лекция лишь освежает эти знания. Если что-то забылось — конспект лекции ([LECTURE-1.md](#)) лежит рядом, и в каждом задании указано, какой его раздел содержит образец кода.

Перед началом. Установка проекта, работа с Git без командной строки и правила сдачи описаны в отдельном документе — «[Организация работы](#)». Задания ниже предполагают, что твоя копия проекта уже заведена, скопирована и запускается (`npm run dev`).

Дедлайн сдачи семинара: первое занятие Темы 3.

Напоминание про Git — в первый и последний раз

В следующих семинарах этих напоминаний не будет, но правила действуют весь курс:

- в `main` напрямую не работаем — на каждое задание своя ветка;
- готовое задание вливается в `main` через merge request (на GitHub — pull request) в **твоей копии** проекта;
- оценочное задание — это открытый MR, который до проверки преподавателем **не** вливается.

Задание 1 (промежуточное). Карточки из данных

Навык: типизация props, циклический и условный рендеринг, CSS-модули. **Ветка:** `feature/board-cards`.

Сейчас `BoardColumn` рендерит две карточки, захардкоженные прямо в JSX, а `BoardCard` умеет показывать только название. Сделай так, чтобы доска строилась из массива данных. Образцы кода — в лекции, разделы «Компоненты» и «Циклический рендеринг».

1. В `App.tsx` заведи массив карточек типа `Card[]` (тип уже готов в `src/types/card.ts`): 3–4 карточки, хотя бы одна с `isDone: true`. Передай массив в `BoardColumn` через props.
2. В `BoardColumn` отрисуй карточки из массива через `.map()`.
3. Добавь компоненту `BoardCard` пропс `isDone` и новый класс в `BoardCard.module.css`: выполненная карточка — полупрозрачная и с зачёркнутым текстом.

Готово, когда:

- ☐ на доске отображаются карточки из твоего массива; выполненные — приглушённые и зачёркнутые;

- ☐ у элементов списка `key={card.id}`, в консоли браузера (`F12`, на macOS `Cmd+Option+I`) нет предупреждения про `key`;
- ☐ новые стили лежат в `BoardCard.module.css`, а не в глобальном CSS и не в атрибуте `style`;
- ☐ в ветке `feature/board-cards` три коммита — по одному на каждый пункт задания, ветка отправлена в твою копию («Publish Branch»).

Задание 2 (промежуточное). Данные с сервера: одна кверя, одна мутация

Навык: `useQuery`, `useMutation`, инвалидация кэша. **Ветка:** `feature/server-cards`.

Сначала Git — влей предыдущую ветку и начни новую:

1. Открой merge request из `feature/board-cards` в `main` **своей копии** и влей его кнопкой «Merge»;
2. В VS Code переключись на `main` (статус-бар → выбери `main`) и нажми «Sync Changes» — локальный `main` получит влитые изменения;
3. Создай от него новую ветку `feature/server-cards`.

Теперь заменим захардкоженный массив настоящими данными. Всё подготовлено заранее: мок-сервер запускается командой `npm run server` во **втором** терминале (первый занят `npm run dev`), а функции `fetchCards` и `createCard` уже лежат в `src/api/cards.ts` — писать `fetch` руками не нужно. Образцы кода — в лекции, разделы «Запросы: `useQuery`» и «Мутации: `useMutation`»; что такое мок-сервер — раздел «Необходимо знать для дальнейшего».

1. В `App.tsx` замени массив из задания 1 на запрос: `useQuery({ queryKey: ["cards"], queryFn: fetchCards })`. Пока данные грузятся (`isLoading`), показывай вместо колонки текст «Загрузка...».
2. Оживи `NewCardForm`: текст поля ввода храни в `useState`; по кнопке «Добавить» вызывай мутацию `useMutation({ mutationFn: createCard })`, а в её `onSuccess` — `invalidateQueries({ queryKey: ["cards"] })`.

Готово, когда:

- ☐ при открытии страницы на мгновение видна «Загрузка...», затем появляются карточки с сервера (те, что в `db.json`);
- ☐ после нажатия «Добавить» новая карточка появляется в колонке сама — без ручного изменения списка в коде;
- ☐ после обновления страницы (`F5`, на macOS `Cmd+R`) добавленные карточки остаются на месте — они действительно сохранены на сервере;
- ☐ захардкоженного массива карточек в коде больше нет;
- ☐ в ветке `feature/server-cards` два коммита — по одному на пункт, ветка отправлена в твою копию.

Задание 3 (оценочное). Тикет KAN-1

Это задание оформлено как задача из трекера — читай его как тикет, который тебе назначили. Перед началом влей `feature/server-cards` в `main` тем же способом, что и в задании 2, и создай ветку `feature/hide-done-filter`.

KAN-1: Скрывать выполненные карточки

Пользователи жалуются: выполненные карточки загромождают доску. Нужен переключатель, который их прячет. Это настройка текущей вкладки — сервер про неё знать не должен.

Критерии приёмки:

1. В `src/store/` создан стор Zustand с полем `hideDone` (по умолчанию `false`) и экшеном `toggleHideDone`;
2. В шапке доски (`BoardHeader`) появилась кнопка: клик переключает `hideDone`, а надпись на кнопке отражает состояние — «Скрыть выполненные» / «Показать все»;
3. При включённом фильтре выполненные карточки не отображаются в колонке; при выключенном — видны все;
4. `BoardColumn` читает `hideDone` из стора через селектор: `useBoardFiltersStore((state) => state.hideDone)`;
5. Отфильтрованный список нигде не хранится — он вычисляется при отрисовке из данных `useQuery` и значения `hideDone`; карточки с сервера в стор не копируются.

Образец кода — в лекции, раздел «Zustand», пример со стором фильтра: он решает почти эту же задачу.

Сдача: отправь ветку `feature/hide-done-filter` в свою копию («Publish Branch») и открой merge request с описанием: что сделано и как проверить (два-три предложения). Этот MR **не вливай** — его проверяет преподаватель. Приёмка — по зачётной системе курса: все критерии выполнены → зачтено, иначе замечания в MR и один круг доработки до дедлайна (подробнее — в [«Организации работы»](#)).

Предостережение

Ошибка, которая обнуляет смысл задания: скопировать карточки с сервера в стор Zustand или в `useState`. Источник истины для карточек один — кэш React Query. В сторе лежит только `hideDone` — одно булево поле.

Частые ошибки — проверь себя перед сдачей

- Индекс массива в `key` вместо `card.id`;
- Список карточек скопирован в `useState` или в стор Zustand — два источника истины, данные разъезжаются;
- Список после мутации обновляется вручную вместо `invalidateQueries`;

- Подписка на весь стор (`useBoardFiltersStore()` без селектора) — компонент перерисовывается при любом изменении стора;
- Merge request по ошибке открыт в репозиторий курса, а не в свою копию (ловушка форка на GitLab) — проверяй target-проект при создании;
- Работа в `main` напрямую и один коммит «всё сразу» — Git-часть задания не засчитывается.